

Technical Assessment: 3D Interactive Jewellery Display

Develop using A-Frame Framework only.

Constraint: Do NOT use starter templates that pre-package interaction logic (e.g., "Interactable" components from a library). You must write the interaction and state management code yourself.

1. Project Brief

Scenario:

You have been tasked with building a high-end interactive product display for a jewellery brand. The client wants a "Jewelry Box" experience where users can inspect multiple rings. Because this is a luxury product, the visual fidelity (lighting and materials) is just as important as the interaction.

The Task:

Create a 3D web scene featuring a jewellery box containing at least **3 distinct ring models** arranged in a layout inside the box.

Core Requirements:

- Visual Fidelity (Lighting & Materials):**
 - The scene must utilize physically based rendering (PBR) or high-quality lighting.
 - The rings must look metallic. As the rings move or rotate, reflections and specular highlights must be visible and dynamic. Flat, unlit colors are not acceptable.
 - Use an Environment Map (HDRI) or dynamic lights to achieve this look.
- Initial State:**
 - The scene loads with the rings sitting stationary inside the box.
- Focus Interaction:**
 - When a user clicks or taps a specific ring:
 - The ring must animate smoothly upwards and forwards (towards the camera/center).
 - The ring must scale up slightly for better visibility.
 - A "Close" button (or UI element) must appear.

4. **Return Interaction:**
 - When the "Close" button is clicked:
 - The ring must animate back to its **exact original position** inside the box.
 - The ring must scale back down to its original size.
 - The UI must disappear.
5. **State Management:** The user must **not** be able to click other rings while one ring is currently focused or mid-animation.

Advanced Requirements:

Implement this interaction logic using an **Entity-Component-System (ECS)** architecture (e.g., A-Frame or custom ECS) rather than standard Object-Oriented programming.

- **Continuous Drag-to-Rotate:** The custom component must enable the user to rotate the loaded 3D model on its Y-axis (Yaw) via a simple drag gesture using a mouse (desktop) or a single finger (mobile touch).
- **Constraint 1: Continuous Rotation:** The rotation must be cumulative and continuous. It must not reset to a starting angle every time the user starts a new drag or tap-and-move sequence.
- **Constraint 2: Y-Axis Only:** The component must control only the Y-axis rotation, leaving the X and Z axes untouched.
- **Constraint 3: Responsiveness:** The rotation speed should be proportional to the distance of the drag.

3. Scoring Rubric (Total: 100 Points)

Passing Score: 70/100

Senior/Advanced Score: 90+/100

Section A: Core Mechanics & Visual Fidelity (35 Points)

Focus: Smoothness and precision.

Criteria	Points	Success Indicators
Lighting & Realism	10	Critical: Uses Environment Maps (HDRI) or dynamic lighting. Metals look metallic. Reflections shift correctly as the object moves.

Animation	10	Uses easing functions (e.g., Cubic, Elastic) rather than linear movement. No visual snapping or jitter.
Position Accuracy	10	Critical: When returning, the ring goes back to its specific, original slot. (Fail if all rings return to the world origin 0,0,0).
UI & Scale Logic	5	Ring scales correctly; "Close" button appears/disappears at correct times.

Section B: State Management & Logic (15 Points)

Focus: Bug prevention and edge cases.

Criteria	Points	Success Indicators
The "Busy" Check	5	Critical: If the user clicks Ring A, and immediately clicks Ring B while A is moving, the app ignores the second click.
Input Isolation	5	Tapping the "Close" UI does not trigger a click on the 3D scene background or other models.
State Reset	5	After closing a ring, the system fully resets. The user can immediately open the same ring again without issues.

Section C: Advanced Interaction with ECS Architecture (40 Points)

Focus: Implementing complex interactions using a modular ECS architecture.

Criteria	Points	Success Indicators
Drag Implementation	15	Dragging left/right rotates the model (Touch/Mouse events used, not a UI slider).

Rotation Continuity	15	The rotation must be smooth and continuous. When a user stops dragging and starts again, the rotation should pick up exactly where it left off (it should not snap back to 0°).
Cleanup	10	When the ring is returned to the box, its rotation must automatically reset to face forward (it should not land sideways or at an odd angle).

Section D: Code Quality (10 Points)

Focus: Cleanliness.

Criteria	Points	Success Indicators
Readability	5	Clear variable naming (e.g., <code>focusedRing</code> , <code>originalPos</code>) vs vague naming (<code>obj</code> , <code>x</code> , <code>temp</code>).
Originality	5	Logic is written by the candidate, not imported from a "DragAndDrop" library template.

Resources:

1. <https://aframe.io/aframe/>
2. <https://aframe.io/aframe-school/>
3. <https://aframe.io/docs/1.7.0/introduction/entity-component-system.html>
4. <https://aframe.io/docs/1.7.0/introduction/writing-a-component.html>
5. <https://aframe.io/docs/1.7.0/introduction/interactions-and-controllers.html>